

視窗應用程式開發與常用 元作介紹 (1)

李東霖 博士

國立臺灣海洋大學 電機工程學系

2025 Fall semester

常用元件介紹(1)

常用元件介紹

- 通用元件

1. Form
2. LinkLabel
3. TextBox
4. ListBox
5. CheckBox
6. RadioButton
7. ComboBox
8. PictureBox

- 容器

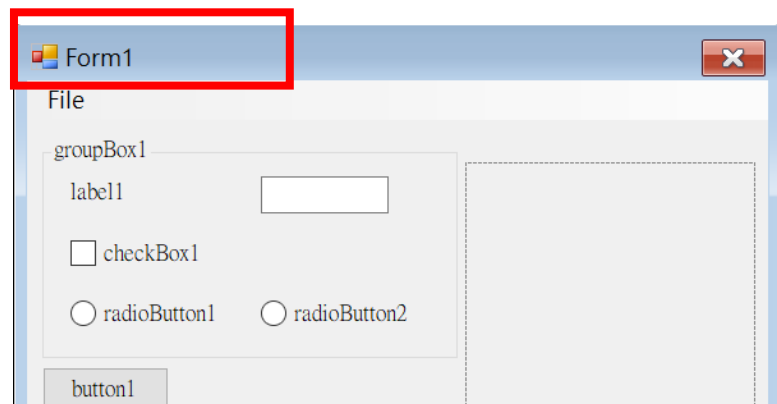
1. GroupBox

- 對話方塊

1. OpenFileDialog
2. SaveFileDialog

Form元件常用屬性

- 常用屬性
 - 外觀
 - BackColor : 背景顏色
 - ForeColor : 文字顏色
 - Font : 字型
 - FormBorderStyle : 視窗大小調整方式
 - Text : 視窗標題顯示文字內容
 - 配置
 - AutoScroll : 當視窗小於配置內容時是否自動加捲動軸
 - StartPosition : 視窗預設出現位置
 - Size : 視窗預設大小
 - WindowState : 視窗啟動時大小
 - 視窗樣式
 - MaximizeBox : 是否要使用視窗放大鈕
 - MinimizeBox : 是否要使用視窗縮小鈕
 - ControlBox : 是否要使用控制按鈕
 - TopMost : 是否最上層顯示



視窗顯示技巧(1) – 開啟時全螢幕阻擋

- 方法一：於設計頁介面點選 **Form** 物件設定以下屬性：
 1. `FormBorderStyle` 設為 `None`，去掉外框。
 2. `WindowState` 設為 `Maximized`，視窗最大化。
 3. `TopMost` 為設 `true`，最上層。
- 方法二：使用程式碼設定屬性
 - 於 `InitializeComponent()` 之後加入：

```
public Form1()  
{  
    InitializeComponent();  
    this.FormBorderStyle = FormBorderStyle.None;  
    this.WindowState = FormWindowState.Maximized;  
    this.TopMost = true;  
}
```

視窗顯示技巧(2) – 開啟時全螢幕阻擋, 切換時縮小

1. 如同前面，但TopMost 為false
2. 在Form 的 Deactivate 事件加入以下內容

```
private void Form1_Deactivate(object sender, EventArgs e)
{
    this.WindowState = FormWindowState.Minimized;
}
```

LinkLabel元件常用屬性

- 常用屬性

- 外觀

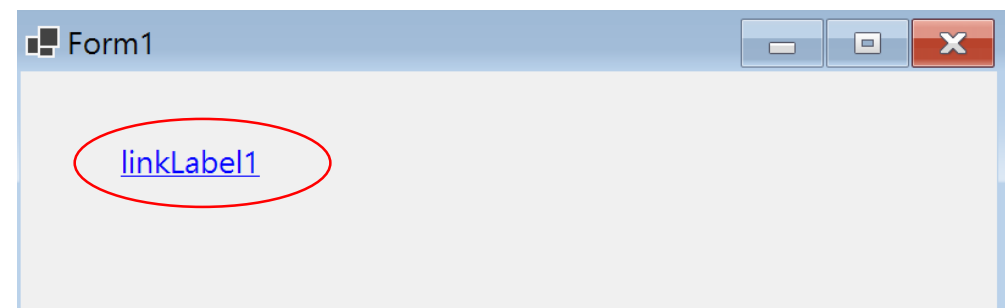
- BackColor : 背景顏色
 - LinkColor : 未訪問過的連結文字顏色
 - VisitedLinkColor : 訪問過的連結文字顏色
 - ActiveLinkColor : 正在訪問的連結文字顏色
 - Font : 字型
 - Text : 顯示文字內容
 - TextAlign : 文字對齊方式

- 行為

- Enabled : 是否可使用
 - Visible : 是否顯示

- 配置

- AutoSize : 自動元件大小調整
 - Size : 元件預設大小

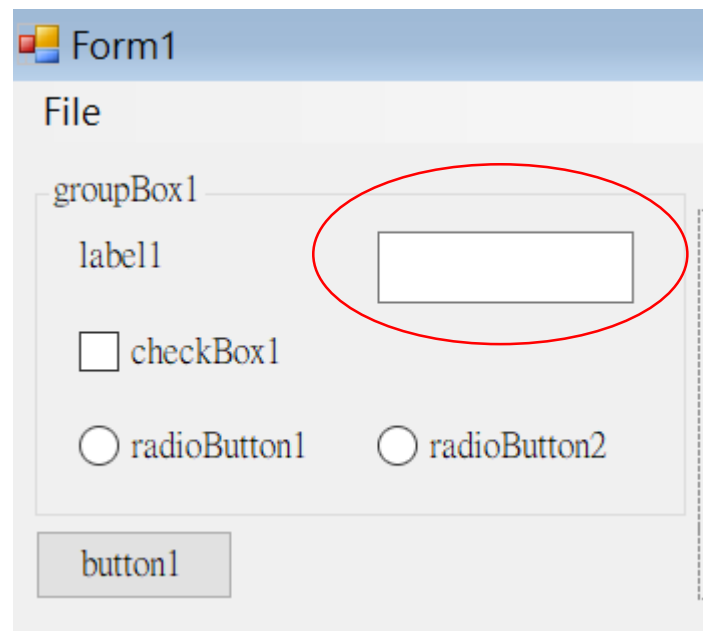


```
using System.Diagnostics;
```

```
private void linkLabel1_LinkClicked(object sender, LinkLabelLinkClickedEventArgs e)
{
    var processStartInfo = new ProcessStartInfo
    {
        FileName = @"https://www.microsoft.com",
        UseShellExecute = true
    };
    System.Diagnostics.Process.Start(processStartInfo);
}
```

TextBox元件常用屬性

- 常用屬性
 - 外觀
 - BackColor : 背景顏色
 - ForeColor : 文字顏色
 - Font : 字型
 - ScrollBars : 是否使用捲動軸
 - Text : 顯示文字內容
 - TextAlign : 文字對齊方式
 - 行為
 - Enabled : 是否可使用
 - MaxLength : 最大輸入長度
 - Multiline : 是否允許多行
 - ReadOnly : 是否唯讀
 - Visible : 是否顯示
 - 配置
 - Size : 元件預設大小

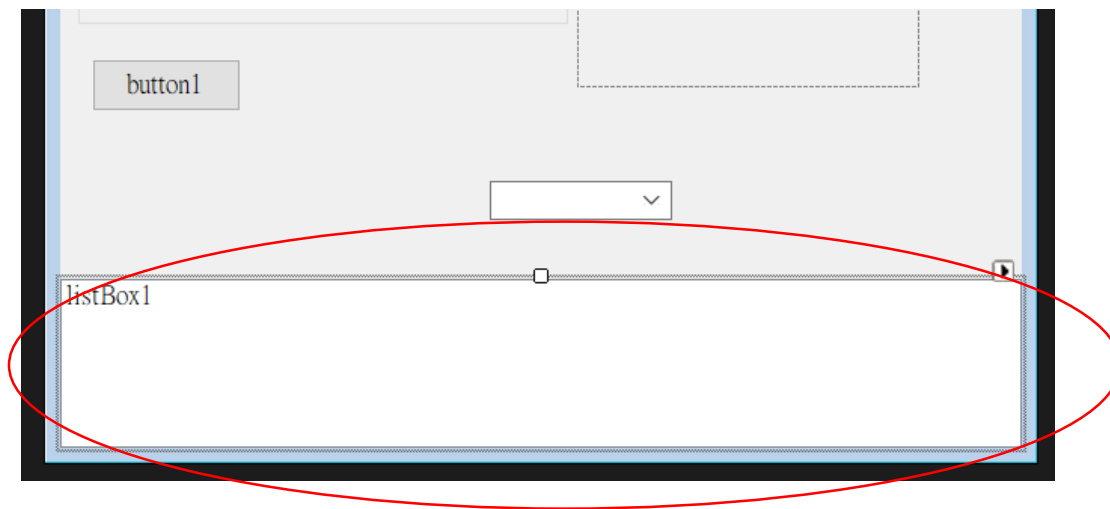


```
int a = Convert.ToInt32(textBox1.Text);  
double f = Convert.ToDouble(textBox1.Text);
```

Convert.(To資料型態)是C#專門用來做資料型態轉換的方法

ListBox元件常用屬性

- 常用屬性
 - 外觀
 - BackColor : 背景顏色
 - ForeColor : 文字顏色
 - Font : 字型
 - TextAlign : 文字對齊方式
 - 行為
 - Enabled : 是否可使用
 - Visible : 是否顯示
 - 配置
 - Size : 元件預設大小
 - 資料
 - Item : 儲放內容

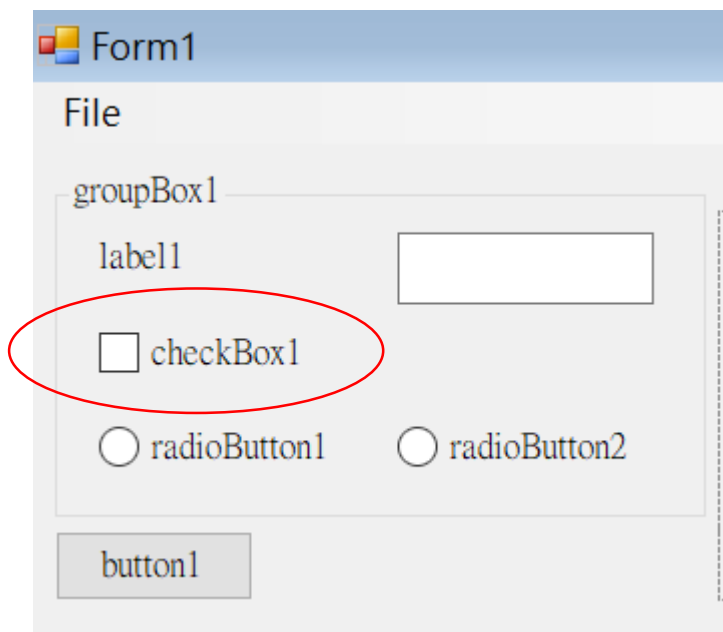


```
listBox1.Items.Add( "字串" );  
string s = listBox1.Items[0].ToString();  
listBox1.Items.Clear();
```

`string`為C#用來儲存字串的物件;
`.ToString()`是C#專門用來將內容輸出成字串的方法;

CheckBox元件常用屬性

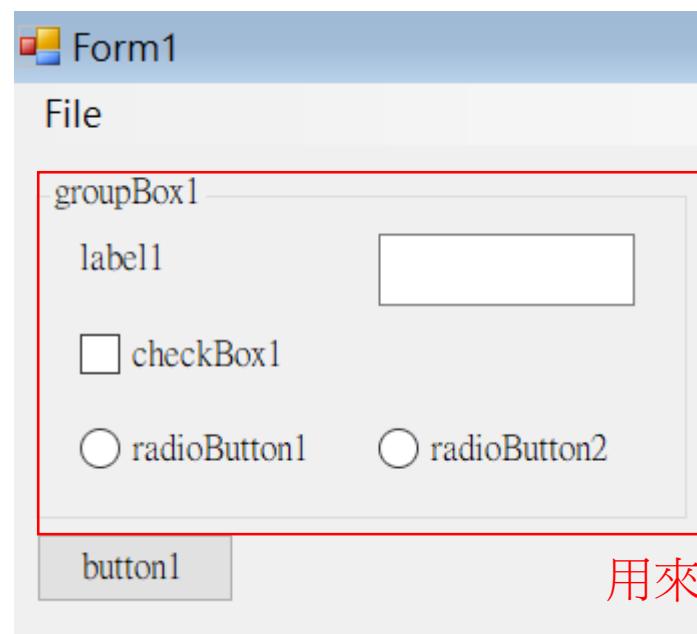
- 常用屬性
 - 外觀
 - BackColor : 背景顏色
 - ForeColor : 文字顏色
 - Checked : 是否勾選
 - Font : 字型
 - Text : 顯示文字內容
 - 行為
 - Enabled : 是否可使用
 - Visible : 是否顯示
 - 配置
 - Size : 元件預設大小



```
if (checkBox1.Checked == true)
    label1.Text = "checked";
```

GroupBox元件常用屬性

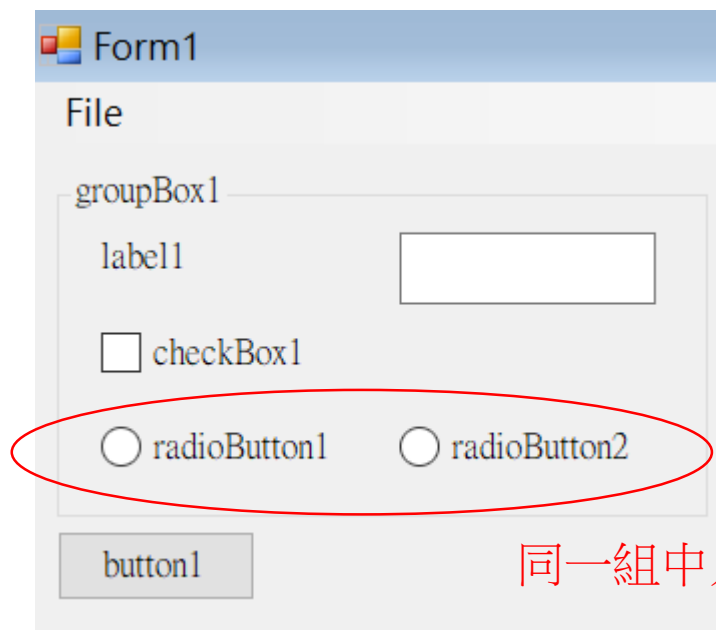
- 常用屬性
 - 外觀
 - BackColor : 背景顏色
 - ForeColor : 文字顏色
 - Checked : 是否勾選
 - Font : 字型
 - Text : 顯示文字內容
 - 行為
 - Visible : 是否顯示
 - 配置
 - Dock : 元件邊緣的連接方式
 - Size : 元件預設大小



用來做元件分組

RadioButton元件常用屬性

- 常用屬性
 - 外觀
 - BackColor : 背景顏色
 - ForeColor : 文字顏色
 - Checked : 是否勾選
 - Font : 字型
 - Text : 顯示文字內容
 - 行為
 - Enabled : 是否可使用
 - Visible : 是否顯示
 - 配置
 - Size : 元件預設大小

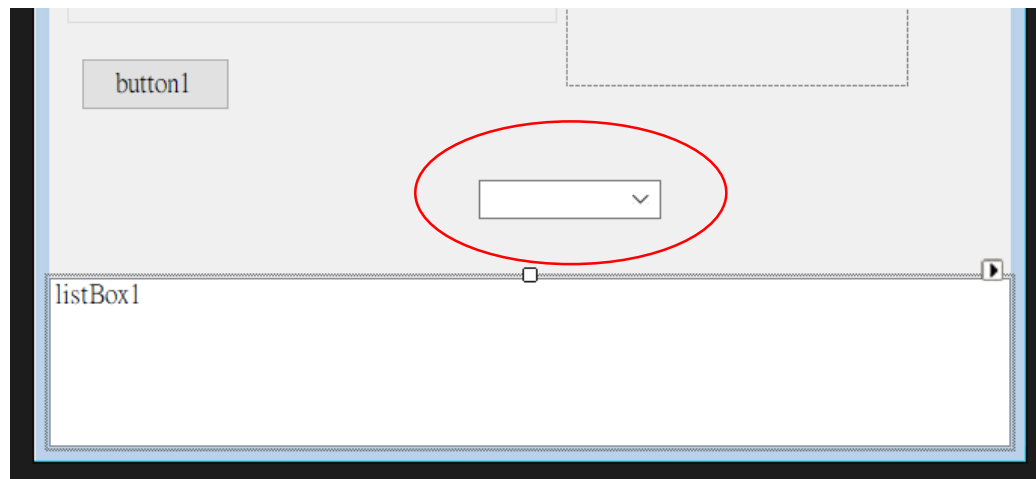


同一組中只有一個會被選擇

```
if (radioButton1.Checked == true)
    label1.Text = "1 is checked";
else if (radioButton2.Checked == true)
    label1.Text = "2 is checked";
```

ComboBox元件常用屬性

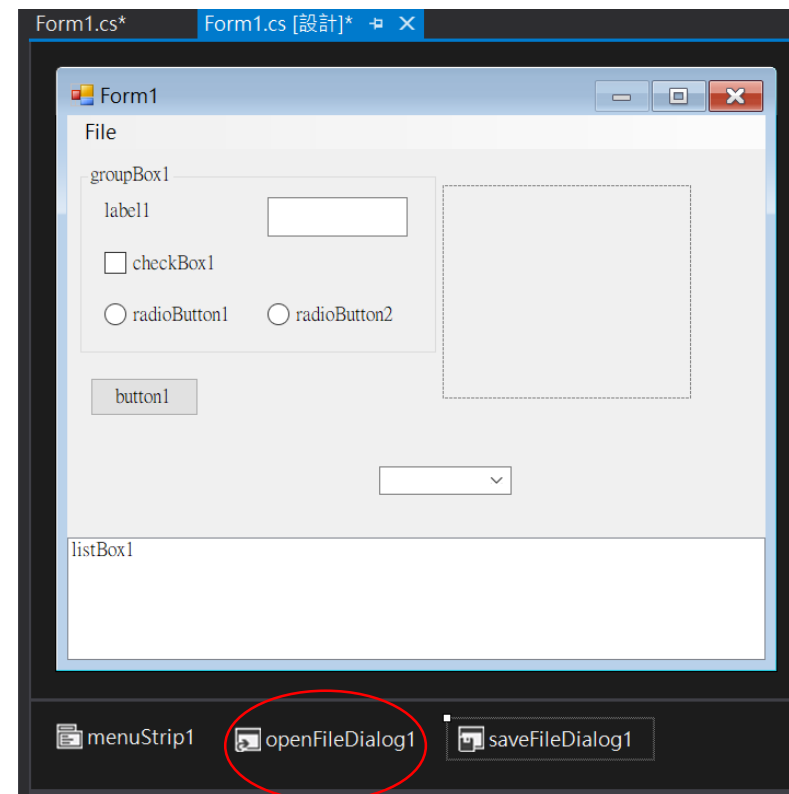
- 常用屬性
 - 外觀
 - BackColor : 背景顏色
 - ForeColor : 文字顏色
 - Font : 字型
 - TextAlign : 文字對齊方式
 - 行為
 - Enabled : 是否可使用
 - Visible : 是否顯示
 - 配置
 - Size : 元件預設大小
 - 資料
 - Item : 儲放內容



```
comboBox1.Items.Add( "字串");  
string s = comboBox1.Items[0].ToString();  
comboBox1.Items.Clear();
```

OpenFileDialog元件常用屬性與應用

- 常用屬性
 - 外觀
 - Title：顯示在對方框上標題的字串
 - 行為
 - Filter：檔案篩選條件
 - Multiselect：是否可選取多個檔案



```
if (openFileDialog1.ShowDialog() == System.Windows.Forms.DialogResult.OK){  
    string theFile = openFileDialog1.FileName; //取得檔名  
    Encoding enc = Encoding.GetEncoding("utf-8"); //設定檔案的編碼  
    textBox1.Text = System.IO.File.ReadAllText(theFile, enc); //以指定的編碼方式讀取檔案  
}
```

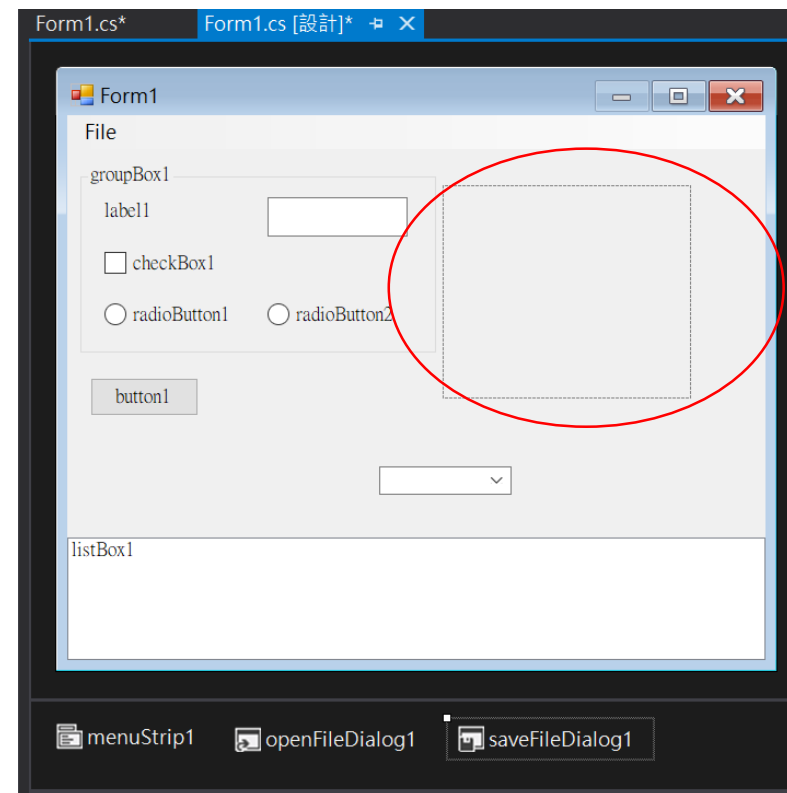
多選檔案的範例

```
private void button1_Click(object sender, EventArgs e)
{
    OpenFileDialog ofd = new OpenFileDialog();
    ofd.InitialDirectory = @"C:\"; //設定初始路徑
    ofd.Filter = "Excel檔案(*.xls)|*.xls|Csv檔案(*.csv)|*.csv|所有檔案(*.*)|*.*"; //設定“另存為檔案型別”或“檔案型別”
    框中出現的選擇內容
    ofd.FilterIndex = 2; //設定預設顯示檔案型別為Csv檔案(*.csv)|*.csv
    ofd.Title = "開啟檔案"; //獲取或設定檔案對話方塊標題
    ofd.RestoreDirectory = true;////設定對話方塊是否記憶上次開啟的目錄

    ofd.Multiselect = true; //設定多選
    if (ofd.ShowDialog() == DialogResult.OK)
    {
        for (int i = 0; i < ofd.FileNames.Length; i++)
        {
            textBox1.Text += ofd.FileNames[i] + "\r\n"; //輸出一個路徑回車換行
        }
        for (int i = 0; i < ofd.SafeFileNames.Length; i++)
        {
            textBox1.Text += ofd.SafeFileNames[i] + "\r\n";
        }
    }
}
```

PictureBox元件常用屬性與應用

- 常用屬性
 - 外觀
 - Image：顯示於PictureBox的影像
 - 行為
 - Enabled：是否可使用
 - SizeMode：圖像的放置位置與大小
 - Visible：是否顯示
 - 配置
 - Size：元件預設大小



```
if (openFileDialog1.ShowDialog() == System.Windows.Forms.DialogResult.OK){  
    pictureBox1.Image = Image.FromFile(openFileDialog1.FileName);  
}
```

SaveFileDialog元件常用屬性與應用

```
private void button1_Click(object sender, EventArgs e)
{
    SaveFileDialog sfd = new SaveFileDialog();
    sfd.Filter = "文字檔案(*.txt)|*.txt|所有檔案|*.*"; //設定檔案型別
    sfd.FileName = "儲存"; //設定預設檔名
    sfd.DefaultExt = ".txt"; //設定預設格式 (可以不設)
    sfd.AddExtension = true; //設定自動在檔名中新增副檔名
    if (sfd.ShowDialog() == DialogResult.OK)
    {
        textBox1.Text = "FileName:" + sfd.FileName + "\r\n";
        using (System.IO.StreamWriter sw = new System.IO.StreamWriter(sfd.FileName))
        {
            sw.WriteLineAsync("今天是個好天氣");
        }
    }
    MessageBox.Show("ok");
}
```

實習時間

Visual Studio 和 GitHub串連

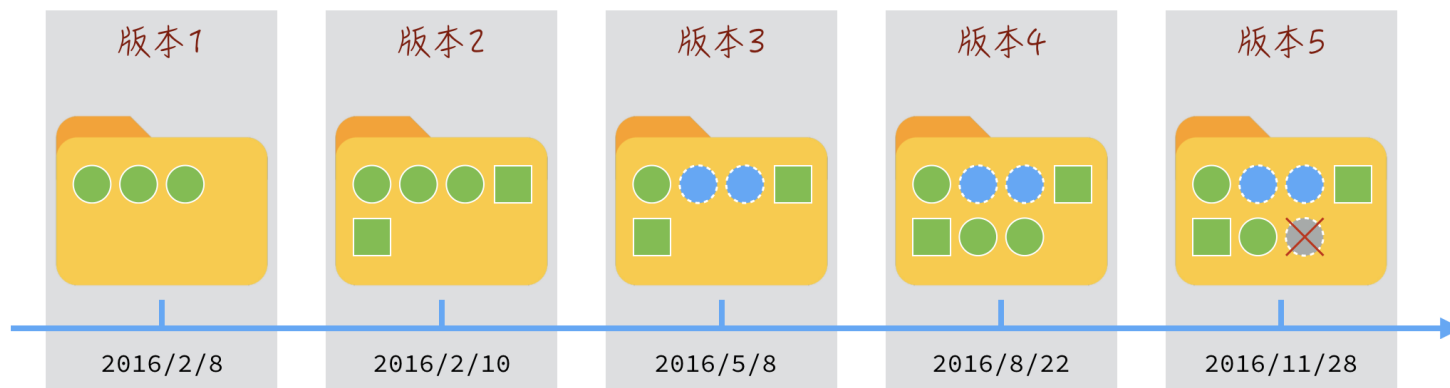
Git與GitHub差別？

- **Git** 是一種版本控制工具。用來追蹤檔案的新增和刪除，提供程式碼修改的歷史記錄與回朔。由於可分散式在多台電腦同時使用並在連網時進行同步管理，所以讓協同開發變得容易。
 - 不論有沒有網路時都可以使用
- **GitHub**是以 **Git** 為版本控制軟體的線上軟體原始碼代管服務平台，跟 **GitLab**、**Bitbucket** 類似。**GitHub** 允許個人或組織建立 repository 和儲存程式碼外，也提供共同軟體開發功能，允許使用者追蹤其他人的專案和提出評論，對於大型專案的開發非常有幫助。
- 簡單來說，**Git**與**GitHub**為現代化程式開發最重要的軟體與管理服務，前者為分散式版本控制的軟體，後者則是以**Git** 作為版本控制之原始碼代管服務平台。

為什麼要學Git?

- **Git** 已經是軟體開發者必備的技能了，幾乎每天都需要用 **Git** 進行版本控管，面對主管、同事或是客戶時，可以更容易地去回想和說明軟體在不同版本之間的差異，新增了什麼功能，解決了什麼 **bug**，當然，如果是大型專案共同開發，也可以更容易地追蹤進度和合併大家的程式碼。

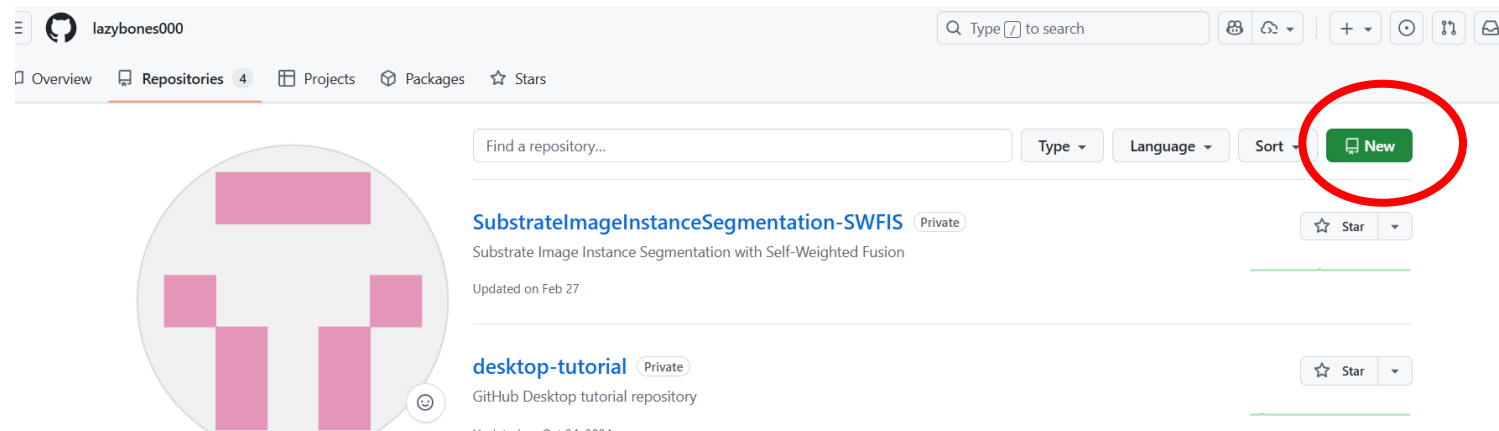
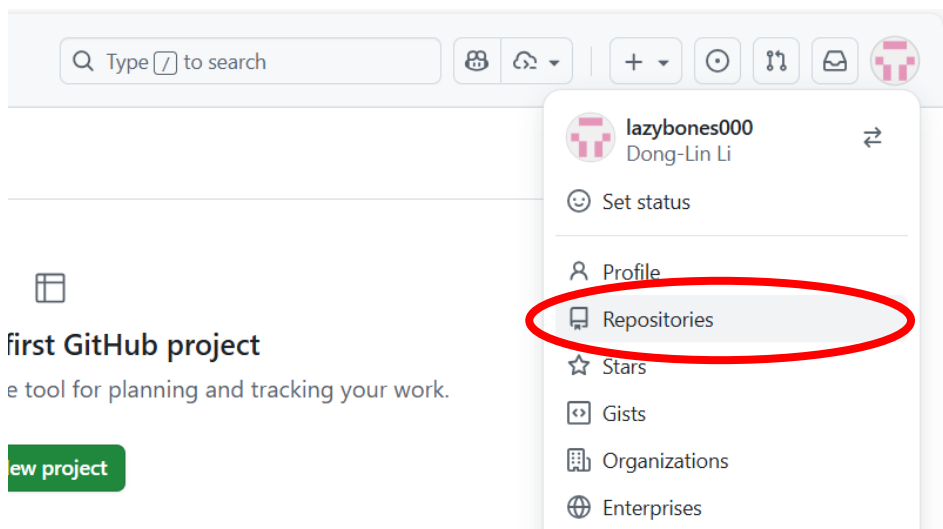
```
▼ 📁 resumes
  > 📁 resume-2016-02-08
  > 📁 resume-2016-02-10
  > 📁 resume-2016-05-08
  > 📁 resume-2016-08-22
  ▼ 📁 resume-2016-11-28
    📄 eddie.md
    📄 john.md
    📄 kao.md
    📄 mary.md
    📄 sherly.md
    📄 tracy.md
  > 📁 resume-bak
  > 📁 resume-for-5xruby
```



Git的優點

- 備份管理容易
- 變更自動紀錄
- 隨時回溯狀態
- 分散式版本控制

在GitHub上創建新的Repository (1)



在GitHub上創建新的Repository (2)

Create a new repository

Repositories contain a project's files and version history. Have a project elsewhere? [Import a repository](#).

Required fields are marked with an asterisk (*).

1 General

Owner *

Repository name *

 lazybones000 / C_test

C_test is available

Great repository names are short and memorable. How about **refactored-octo-eureka**?

Description

0 / 350 characters

2 Configuration

Choose visibility *

Choose who can see and commit to this repository

 Public

Add README

READMEs can be used as longer descriptions. [About READMEs](#)

On ☒

Add .gitignore

.gitignore tells git which files not to track. [About ignoring files](#)

No .gitignore

3 Get started with Copilot (optional)

Describe what you want Copilot to make. After your repository is created, Copilot will open a pull request, work in the background, then request your review when ready. [About Copilot coding agent](#)

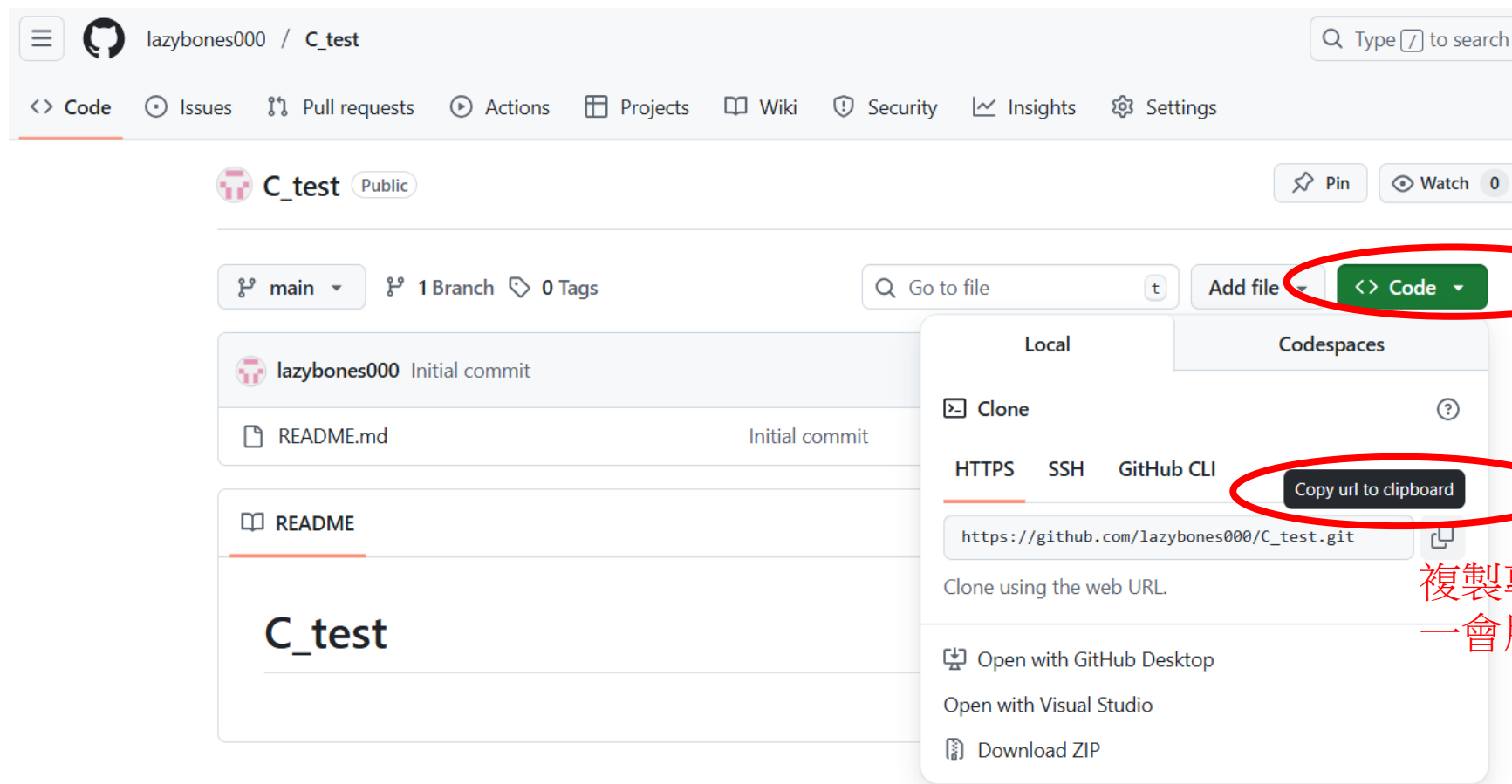
Prompt

Describe what you want Copilot to build

0 / 500 characters

Create repository

創建新的Repository (3)



複製專案網址到剪貼簿
一會用到

透過 Visual Studio 登入 GitHub

建立 Git 存放庫

推送至新的遠端存放庫

- GitHub
- Azure DevOps

其他

- 現有遠端存放庫
- 僅限本機

將本機 Git 存放庫初始化

本機路徑(L)

.gitignore 範本

授權範本

☐ 新增 README.md(R)

建立新的 GitHub 存放庫

帳戶(A)

加入帳戶:

- GitHub 帳戶

擁有者(O)

存放庫名稱(R)

描述(D)

可見度

您可以選擇能夠查看及提交此存放庫的人員。

將您的程式碼推送到 GitHub

建立並推送 取消

Sign in to GitHub
to continue to Visual Studio

Username or email address

英 Password [Forgot password?](#)

Sign in

or

Continue with Google

New to GitHub? [Create an account](#)

[Sign in with a passkey](#)

Two-factor authentication

Enter the code from your two-factor authentication app or browser extension below.

Verify

More options

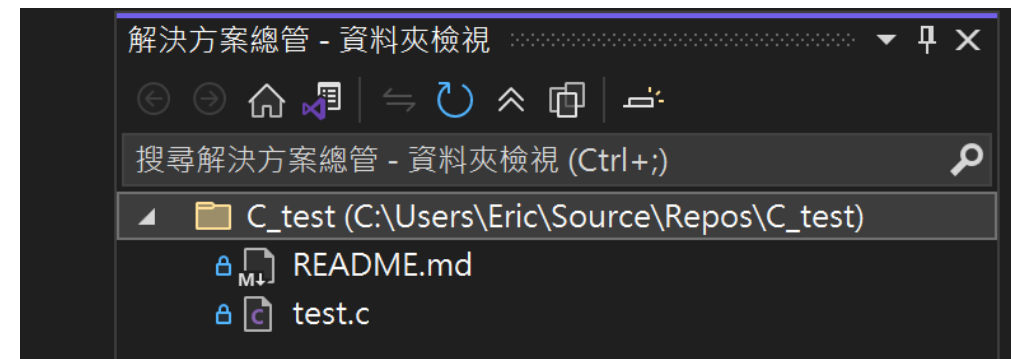
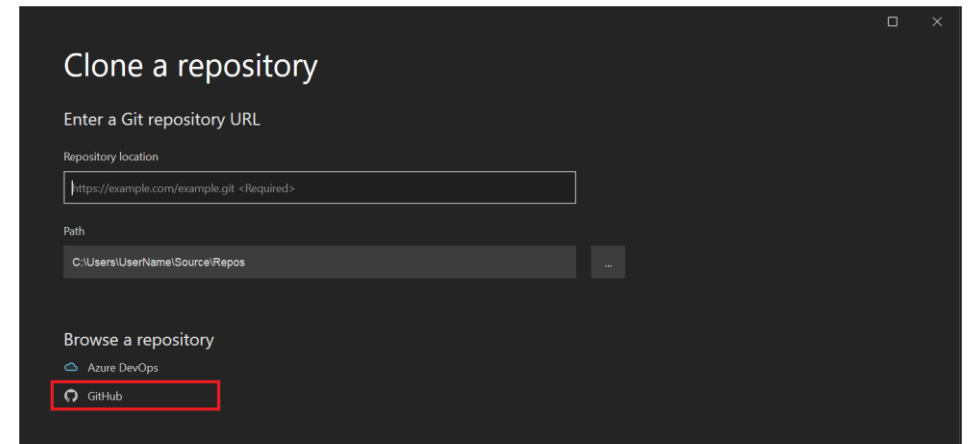
Select user to authorize
Visual Studio

Signed in as lazybones000 [Continue](#)

Use a different account

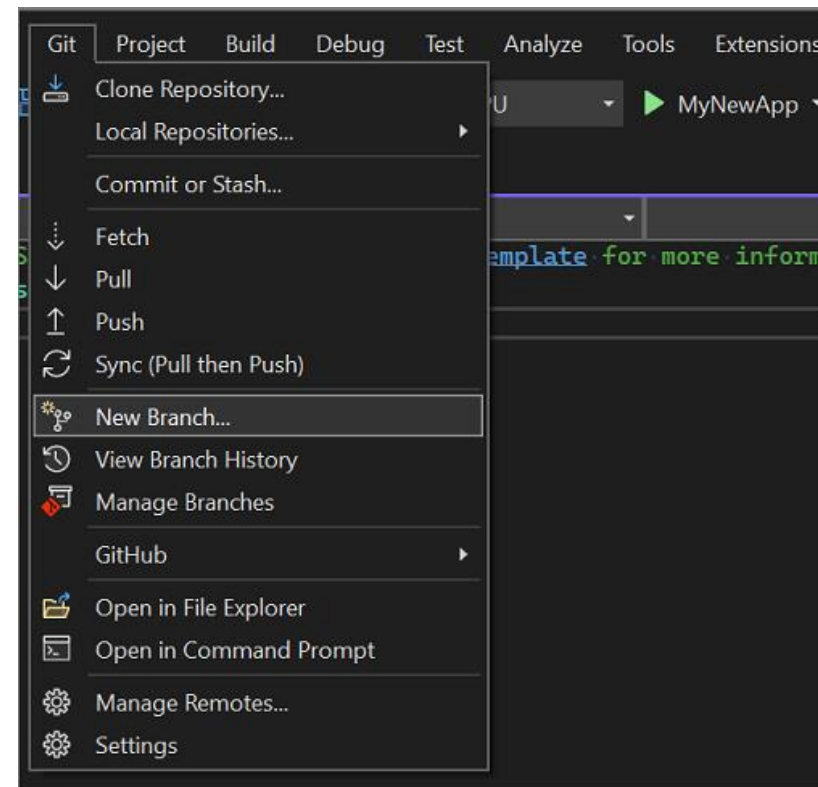
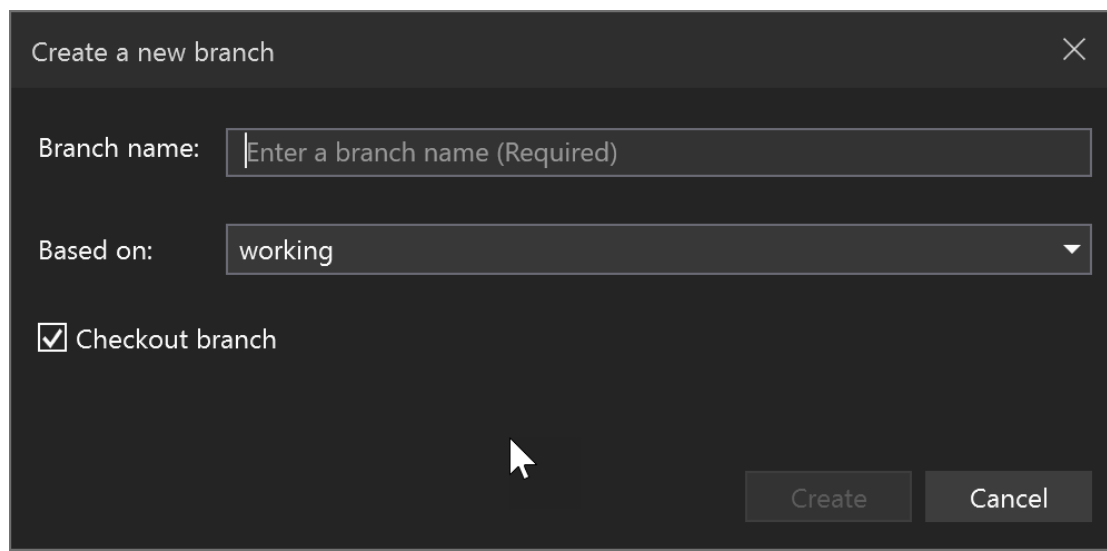
在 Visual Studio 中複製 Github 的存放庫 (Clone repository)

1. 開啟 Visual Studio 建立新專案時選複製存放庫或在建或從 [Git] 功能表中選取 [複製存放庫]。
2. 輸入 Git 存放庫的 URL 與存放位置。
 - URL 為剛複製的專案網址
 - 或透過瀏覽 GitHub 的方式選取
3. Visual Studio 會在存放庫中顯示解決方案清單。選擇您想要載入的解決方案，或在方案總管中開啟 [資料夾檢視]。

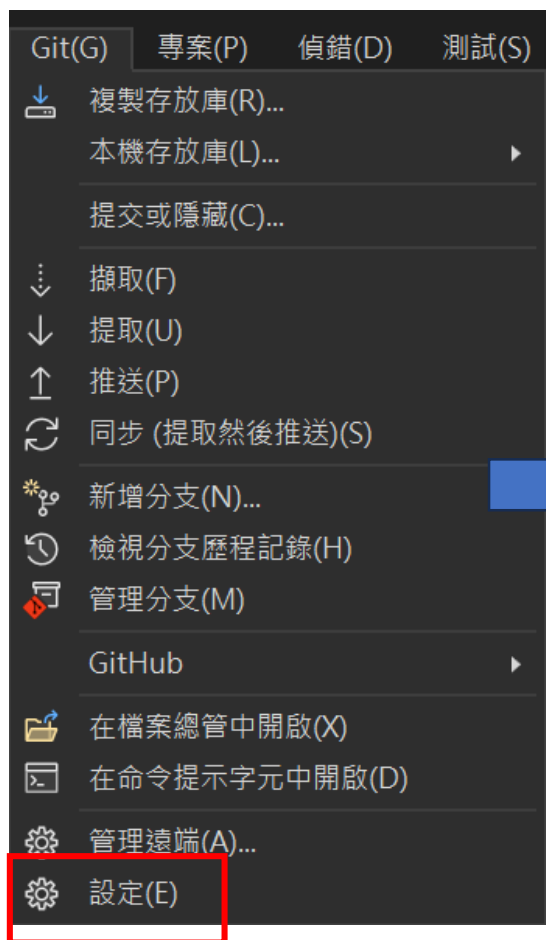


在 Visual Studio 中建立 Git 分支

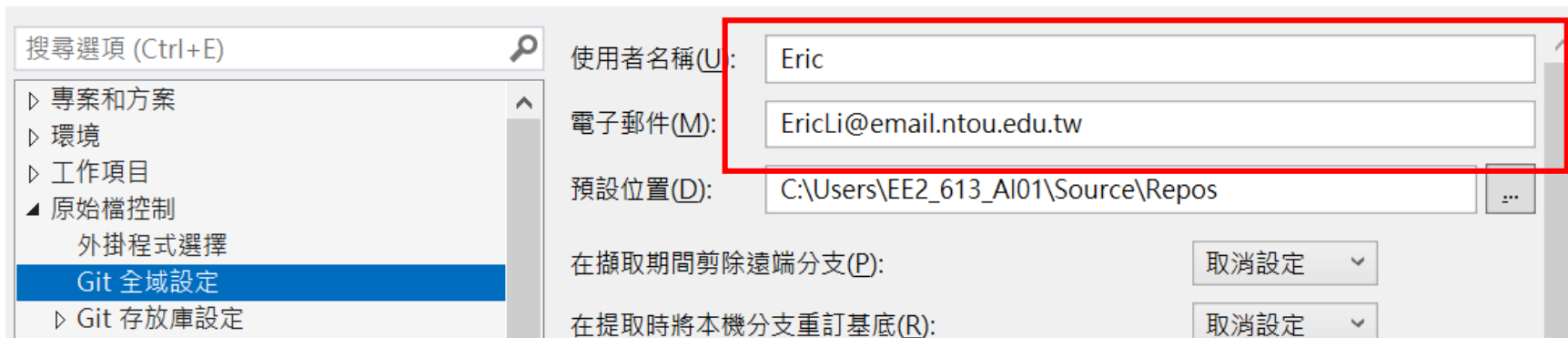
1. 開啟先前建立的或複製的存放庫。
2. 從 [Git] 功能表中，選取 **New Branch**(新增分支)。
3. 在 [建立新的分支] 對話方塊中，輸入分支名稱。
4. 在 [根據] 區段中，使用下拉式清單來選擇您要根據現有的本機分支或遠端分支來建立新分支的基礎。



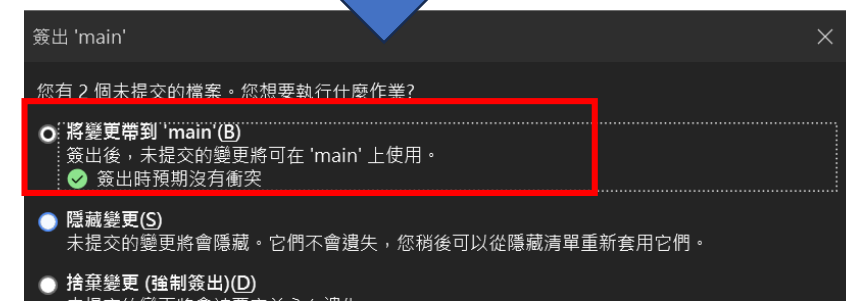
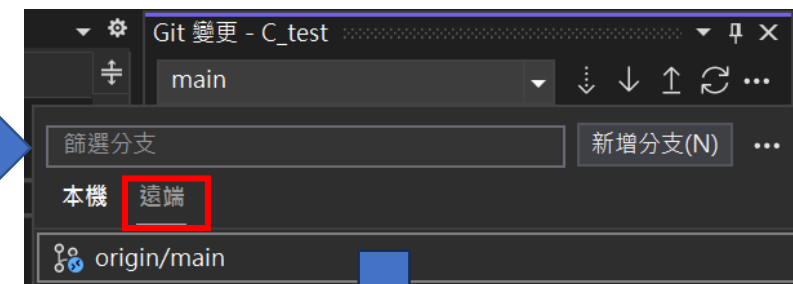
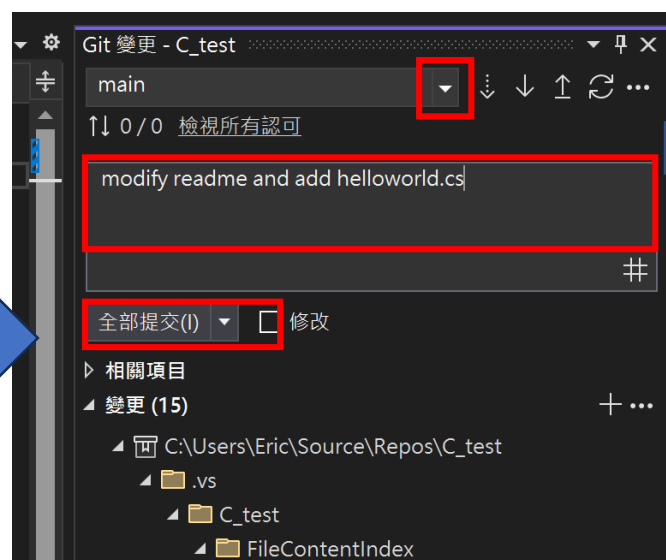
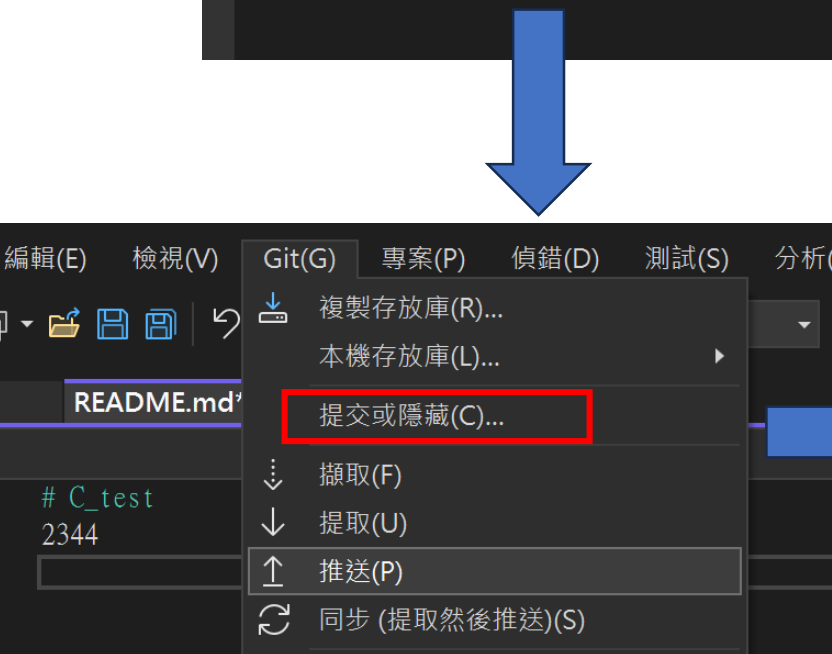
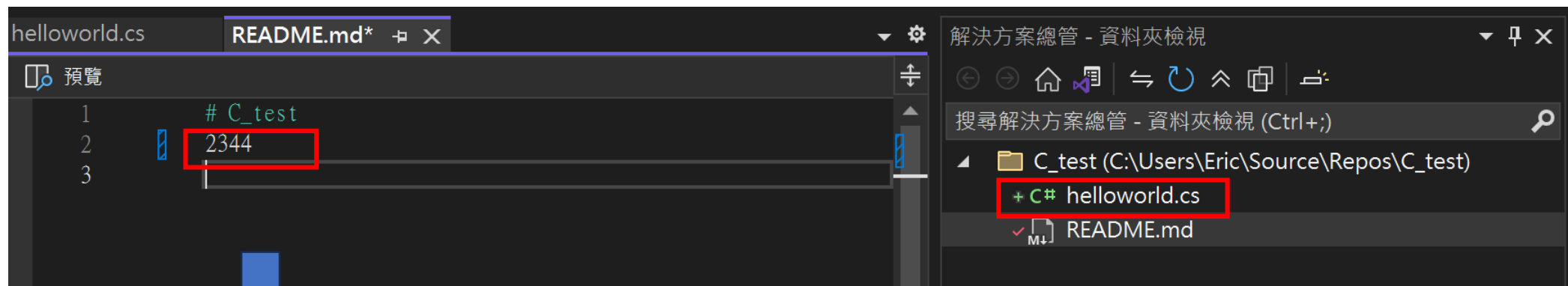
設定使用者名稱



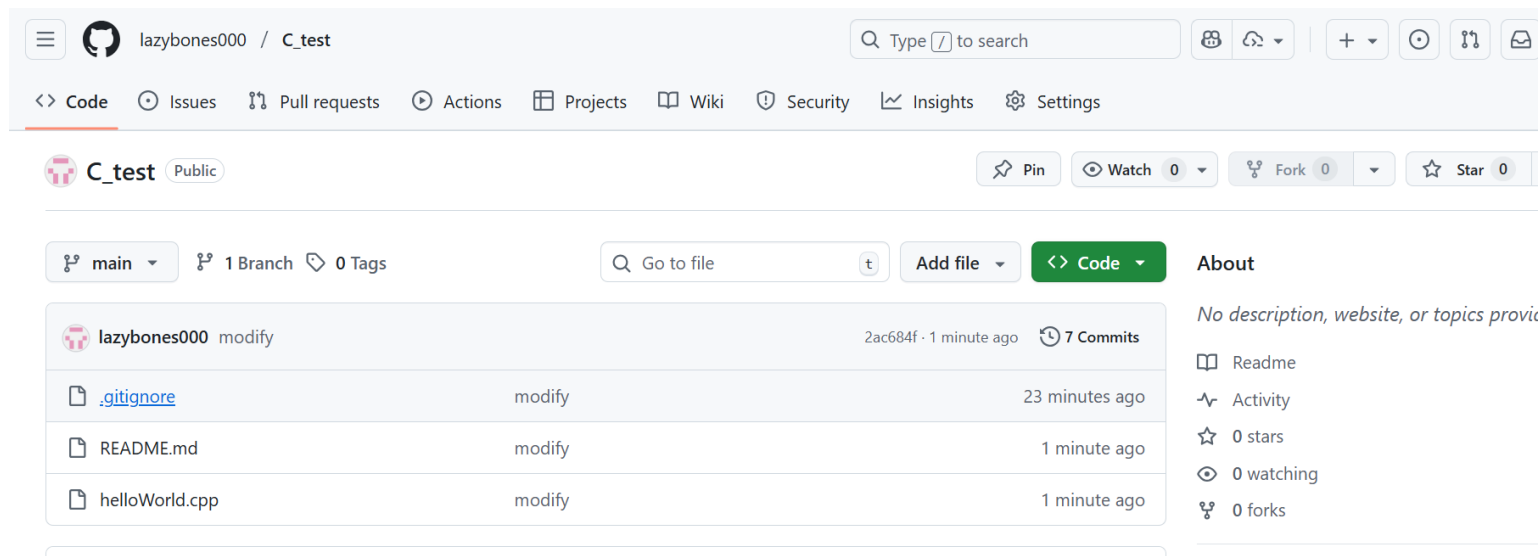
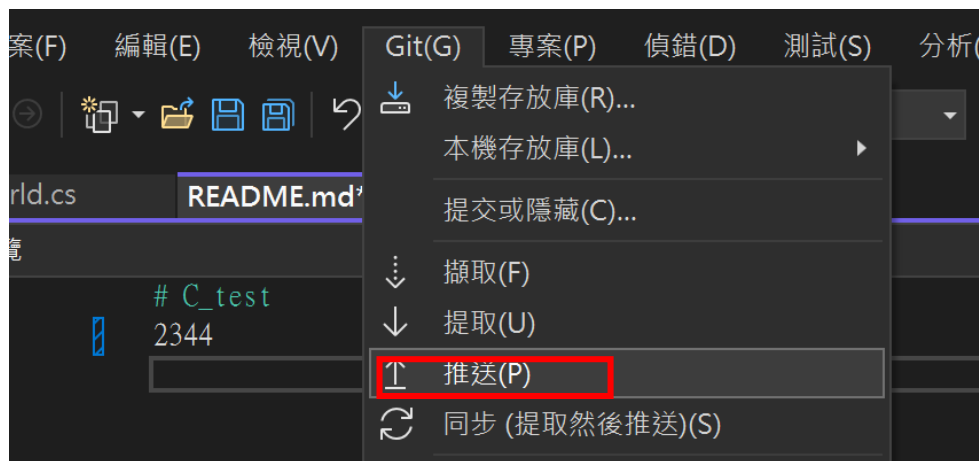
選項



在 Visual Studio 修改 README.md 文字並新增一個檔後提交



在 Visual Studio 中使用 Git 的 Push(推送)



在 Visual Studio 中使用 Git 的 Fetch(擷取)/ Pull(提取)/ Sync(同步)

- Fetch(擷取)的功能在比對本機端Repository與遠端Repository上的檔案內容差異。但不會做任何動作。
- Pull(提取)則是會將遠端Repository修改過的內容下載回本機端
- Sync(同步)則是會將本機端和遠端的Repository進行同步(先Pull再Push)